CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 11

Memory Management Scheme



Memory Management Scheme

Topics to be Covered

1. Memory Management
2. Non-Contiguous Memory Allocation
3. Bare Machine
4. Resident Monitor
5. Multiprogramming with Fixed Partition

Memory Management

- Memory management determines how memory space is allocated to processes.
- It controls the **allocation and deallocation** of memory.
- **Main Categories:**
  - **Non-Contiguous Memory Allocation:**
    - A single logical process or program may be placed in **non-consecutive memory blocks**.
    - Allows flexibility and efficient use of memory.
    - Two main mechanisms:
      i. **Paging (System view)** – memory is divided into fixed-size pages.
      ii. **Segmentation (User view)** – memory is divided based on logical units like code, data, and stack.
    - Both mechanisms help manage **non-contiguous memory allocation**.

Bare Machine (Single User System)

- **Definition:**
  - Simplest memory management method.
  - The **user has complete control** over memory.
  - The operating system has **no control**.
  - Typically used in **single-user or dedicated systems**.

  Advantages:

  (a) **Flexible** – the user can manage memory freely.
  (b) **No hardware support required.**
  (c) **No OS software needed** to control memory.

  Disadvantages:

  - Since OS has no control:
    - No system services (like I/O handling, protection, or scheduling).
    - No resource management.
  - Used only in **user-dedicated systems**, not in multiprogramming environments.

Resident Monitor (Single Process Monitor)

- **Concept:**
  - Simplest form of memory management under OS control.
  - Memory is divided into **two contiguous areas**:
    1. **Resident area (Monitor):** Permanently allocated to the OS.
    2. **User area:** Remaining space used by one program at a time (transient process).

- **Working:**
  - OS (monitor) stays permanently in memory.
  - User processes (transient processes) are loaded one after another and executed one at a time.
- **Limitation:**

  Low processor and memory utilization because:
  - There is **no multiprogramming support**.
  - When a process is waiting for I/O completion, the CPU remains **idle**.
  - Hence, CPU cycles get wasted.

Multiprogramming with Fixed Partition

- **Concept:**

  Introduced to **improve system performance**.
- **Definition:**
  - "More than one process can be in memory at the same time."
  - Only one process is executed by the CPU at a time (in a uniprocessor), while others wait for I/O.
  - The **CPU scheduler** decides which process runs next.
- **Working Principle:**
  - Physical memory is **divided into several fixed-size partitions**.
  - Each partition holds **exactly one process**.
  - If a process finishes or is swapped out, another process is loaded into that partition.
  - Supports **concurrent access** of multiple programs.

Partitioned Memory Management (Contiguous Allocation)

- In this method, **physical memory** is divided into **separate sections called partitions**.
- Each partition can hold **a different process**.
- Memory partitioning can be of **two types:**
  - **Fixed (Static) Partitioning**
  - **Variable (Dynamic) Partitioning**

Fixed (Static) Partitioning

- Partitions are created **at system generation time** (before execution).
- Sizes can differ, but **cannot be changed** once defined.
- Can be implemented **with or without swapping or relocation**.

Problems in Static Partitioning

a. **Partition Selection:** How to select which partition to allocate for a given process.

b. **No Suitable Partition:** What to do when no partition can fit the process size.

Partition Selection Strategies

Strategy	Description	Pros / Cons
First Fit	Allocates the **first free partition** that is large enough for the process.	Simple and fast. May lead to **wasted space** (external fragmentation).
Best Fit	Allocates the **smallest partition** that meets the process's needs.	**Efficient memory use** creates the smallest leftover gap. Slower due to more searching. Causes **internal fragmentation** (unused space within allocated partition).

Internal Fragmentation

- Occurs when a process **does not fully use** the memory allocated to it.
- The **unused portion** inside a fixed partition is wasted.
- This is a **primary problem** in static partitioned systems.

Performance Notes

- **Resident Monitor:** Wastes CPU time during I/O waits.
- **Fixed Partition Multiprogramming:** Improves CPU utilization.
- **Best Fit vs First Fit:** Trade-off between speed and memory efficiency.

मेमोरी मैनेजमेंट स्कीम

शामिल किए जाने वाले विषय

1. मेमोरी मैनेजमेंट
2. नॉन-कंटीगुअस मेमोरी एलोकेशन
3. बेयर मशीन
4. रेजिडेंट मॉनिटर
5. फिक्स्ड पार्टीशन के साथ मल्टीप्रोग्रामिंग

मेमोरी मैनेजमेंट

- मेमोरी मैनेजमेंट यह तय करता है कि प्रोसेस को मेमोरी स्पेस कैसे एलोकेट किया जाता है।
- यह मेमोरी के एलोकेशन और डीएलोकेशन को कंट्रोल करता है।
- **मुख्य श्रेणियां:**
  - **नॉन-कंटीगुअस मेमोरी एलोकेशन:**
    - एक सिंगल लॉजिकल प्रोसेस या प्रोग्राम को नॉन-कोंसीक्युटिव मेमोरी ब्लॉक में रखा जा सकता है।
    - यह फ्लेक्सिबिलिटी (लचीलेपन) और मेमोरी का कुशल उपयोग करने की अनुमति देता है।
    - **दो मुख्य तकनीक:**
      - **i.** पेजिंग (सिस्टम व्यू) – मेमोरी को निश्चित आकार के पेज में बांटा जाता है।
      - **ii.** सेगमेंटेशन (यूज़र व्यू) – मेमोरी को कोड, डेटा और स्टैक जैसी लॉजिकल यूनिट के आधार पर बांटा जाता है।
    - ये दोनों तकनीक नॉन-कंटीगुअस मेमोरी एलोकेशन को मैनेज करने में मदद करते हैं।

बेयर मशीन (सिंगल यूज़र सिस्टम)

- **परिभाषा:**
  - सबसे आसान मेमोरी मैनेजमेंट विधि।
  - यूज़र का मेमोरी पर पूरा कंट्रोल होता है।
  - ऑपरेटिंग सिस्टम का कोई कंट्रोल नहीं होता है।
  - आमतौर पर सिंगल-यूज़र या डेडिकेटेड सिस्टम में इस्तेमाल होता है।

  लाभ:

  (a) फ्लेक्सिबल/लचीलापन – यूज़र मेमोरी को आज़ादी से मैनेज कर सकता है।

  (b) किसी हार्डवेयर सपोर्ट की ज़रूरत नहीं है।

  (c) मेमोरी को कंट्रोल करने के लिए किसी OS सॉफ्टवेयर की ज़रूरत नहीं है।

  हानियाँ:
  - क्योंकि OS का कोई कंट्रोल नहीं होता:
    - कोई सिस्टम सर्विस नहीं (जैसे I/O हैंडलिंग, प्रोटेक्शन, या शेड्यूलिंग)।
    - कोई रिसोर्स मैनेजमेंट नहीं।
  - इसका इस्तेमाल सिर्फ़ यूज़र-डेडीकेटेड सिस्टम में होता है, मल्टीप्रोग्रामिंग एनवायरनमेंट में नहीं।

रेजिडेंट मॉनिटर (सिंगल प्रोसेस मॉनिटर)

- **अवधारणा:**
  - OS कंट्रोल के तहत मेमोरी मैनेजमेंट का सबसे आसान रूप।
  - मेमोरी को दो कंटीगुअस एरिया में बांटा गया है:
    1. रेजिडेंट एरिया (मॉनिटर): OS को स्थायी रूप से एलोकेट किया गया है।
    2. यूज़र एरिया: बची हुई जगह एक समय में एक प्रोग्राम द्वारा इस्तेमाल की जाती है (ट्रांजिएंट प्रोसेस)।
- **कार्यप्रणाली:**
  - OS (मॉनिटर) मेमोरी में स्थायी रूप से रहता है।
  - यूज़र प्रोसेस (ट्रांजिएंट प्रोसेस) एक के बाद एक लोड होते हैं और एक समय में एक ही एग्जीक्यूट होता है।
- **सीमाएं:**
  - कम प्रोसेसर और मेमोरी का इस्तेमाल क्योंकि:
    - इसमें मल्टीप्रोग्रामिंग सपोर्ट नहीं है।
    - जब कोई प्रोसेस I/O पूरा होने का इंतज़ार कर रहा होता है, तो CPU बेकार रहता है।
    - इसलिए, CPU साइकिल बर्बाद होते हैं।

फिक्स्ड पार्टीशन के साथ मल्टीप्रोग्रामिंग

- **अवधारणा:**
  सिस्टम परफॉर्मेंस को बेहतर बनाने के लिए पेश किया गया।
- **परिभाषा:**
  - "एक ही समय में मेमोरी में एक से ज़्यादा प्रोसेस हो सकते हैं।"
  - एक समय में CPU (यूनिप्रोसेसर में) सिर्फ़ एक प्रोसेस को एग्जीक्यूट करता है, जबकि दूसरे I/O का इंतज़ार करते हैं।
  - CPU शेड्यूलर तय करता है कि अगला कौन सा प्रोसेस चलेगा।
- **काम करने का सिद्धांत:**
  - फिजिकल मेमोरी को कई फिक्स्ड-साइज़ के पार्टीशन में बांटा जाता है।
  - हर पार्टीशन में ठीक एक प्रोसेस होता है।
  - अगर कोई प्रोसेस खत्म हो जाता है या उसे स्वैप आउट कर दिया जाता है, तो उस पार्टीशन में दूसरा प्रोसेस लोड किया जाता है।
  - यह कई प्रोग्राम के एक साथ एक्सेस को सपोर्ट करता है।

पार्टीशन्ड मेमोरी मैनेजमेंट (कंटीगुअस एलोकेशन)

- इस तरीके में, फिजिकल मेमोरी को अलग-अलग सेक्शन में बांटा जाता है जिन्हें पार्टीशन कहते हैं।
- हर पार्टीशन एक अलग प्रोसेस को रख सकता है।
- मेमोरी पार्टीशनिंग दो तरह की हो सकती है:
  - फिक्स्ड (स्टैटिक) पार्टीशनिंग
  - वेरिएबल (डायनामिक) पार्टीशनिंग

फिक्स्ड (स्टैटिक) पार्टीशनिंग

- पार्टीशन सिस्टम बनने के समय (एग्जीक्यूशन से पहले) बनाए जाते हैं।
- साइज़ अलग-अलग हो सकते हैं, लेकिन एक बार तय होने के बाद उन्हें बदला नहीं जा सकता।
- इसे स्वैपिंग या रिलोकेशन के साथ या उसके बिना लागू किया जा सकता है।

स्टैटिक पार्टिशनिंग में समस्याएं

a. **पार्टिशन का चुनाव:** किसी दिए गए प्रोसेस के लिए कौन सा पार्टिशन एलोकेट किया जाए, यह कैसे चुनें।
b. **कोई उपयुक्त पार्टिशन नहीं:** जब कोई भी पार्टिशन प्रोसेस के साइज़ के हिसाब से फिट न हो, तो क्या करें।

पार्टीशन चुनने की रणनीतियाँ

रणनीति	विवरण	फायदे / नुकसान
फर्स्ट फिट	पहला फ्री पार्टीशन एलोकेट करता है जो प्रोसेस के लिए काफी बड़ा हो।	सरल और तेज़। इससे जगह बर्बाद हो सकती है (एक्सटर्नल फ्रेगमेंटेशन)।
बेस्ट फिट	सबसे छोटा पार्टीशन एलोकेट करता है जो प्रोसेस की ज़रूरतों को पूरा करता हो।	कुशल मेमोरी उपयोग से सबसे छोटा बचा हुआ गैप बनता है। ज़्यादा सर्चिंग के कारण धीमा। इंटरनल फ्रेगमेंटेशन (एलोकेटेड पार्टीशन के अंदर बिना इस्तेमाल की गई जगह) का कारण बनता है।

इंटरनल फ्रेगमेंटेशन

- यह तब होता है जब कोई प्रोसेस उसे दी गई मेमोरी का पूरी तरह से इस्तेमाल नहीं करता है।
- एक फिक्स्ड पार्टीशन के अंदर का बिना इस्तेमाल किया हुआ हिस्सा बर्बाद हो जाता है।
- यह स्टैटिक पार्टीशन्ड सिस्टम में एक मुख्य समस्या है।

प्रदर्शन संबंधी नोट्स

- **रेजिडेंट मॉनिटर:** I/O वेट के दौरान CPU टाइम बर्बाद करता है।
- **फिक्स्ड पार्टीशन मल्टीप्रोग्रामिंग:** CPU यूटिलाइज़ेशन को बेहतर बनाता है।
- **बेस्ट फिट बनाम फर्स्ट फिट:** स्पीड और मेमोरी एफिशिएंसी के बीच ट्रेड-ऑफ।



PW Web/App: https://smart.link/7wwosivoicgd4
Library- https://smart.link/sdfez8ejd80if